08-eval-experimentation

20. Evaluation Framework

The Evaluation Framework is the trust centerpiece of Helios. Because the LLM agents (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic) make causal-sounding claims about WHY the funnel moved, those claims are worthless unless we can prove the system finds the *right* root cause more reliably than a naive analyst. We prove this with an **offline, labeled benchmark**: we inject synthetic-but-known anomalies into a frozen copy of the GA4 data, run the full Helios pipeline against the perturbed copy, and grade its diagnosis against ground truth we recorded at injection time. The headline contract: **root-cause segment accuracy >= 85% on the labeled benchmark vs <= 45% for the naive baseline**, with **0 hallucinated columns/metrics** and **100% of findings carrying a significance test and a dollar revenue-at-risk**.

20.1 Why offline injection (and not the live data)

`bigquery-public-data.ga4_obfuscated_sample_ecommerce` is a fixed historical export (~2020-11-01 to 2021-01-31). It contains real anomalies, but they are **unlabeled** — we do not know their true root causes, so we cannot grade against them. We therefore construct a *counterfactual*: take a real baseline period, clone it, and surgically perturb one or more `(metric, dimension-segment, time)` cells by a **known amount**, recording the perturbation as ground truth. The pipeline must rediscover what we hid. This is the only way to compute decomposition error and root-cause accuracy with confidence intervals.

20.2 Injection mechanism

Injection operates on a **scenario fixture table** materialized in the eval dataset `helios_eval`. We never mutate the public source. The flow:

1. `warehouse-mcp.run_query` extracts a baseline window into `helios_eval.fct_daily_funnel_base` (the canonical `fct_daily_funnel` grain: one row per `day` x dimension cell with `sessions`, `view_item_sessions`, `add_to_cart_sessions`, `begin_checkout_sessions`, `purchasing_sessions`, `revenue`, `transactions`).
2. A deterministic Python injector (seeded) reads a **scenario spec** (YAML below) and produces `helios_eval.fct_daily_funnel_perturbed` by applying one of two perturbation primitives at the specified `inject_at` date onward:

- **rate perturbation**: multiply a segment's *conversion rate* at a funnel step by `rate_multiplier`. Operationally, for a segment cell we recompute the numerator (e.g. `purchasing_sessions`) as `round(sessions * base_rate * rate_multiplier)` while holding `sessions` (the volume/weight `w_i`) fixed. This changes `r_i` only -> ground-truth = **rate-change**.
- **volume/mix perturbation**: multiply a segment's `sessions` by `volume_multiplier` while holding that segment's per-step rates fixed, then renormalize so total sessions is conserved within tolerance. This changes `w_i` only -> ground-truth = **mix-shift**.

3. The injector writes a **ground-truth label record** to `helios_eval.labels` capturing: scenario_id, anomaly_type (`mix` | `rate` | `mixed` | `none`), affected metric, affected segment key(s) and dimension, `inject_at`, the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on (see Foundation CORE ALGORITHM), and the **true dollar-at-risk** = `(counterfactual_revenue_without_perturbation - perturbed_revenue)` summed over the post-injection window.

Because perturbations are applied to aggregates with conserved totals, the analytic mix/rate/interaction split is exact and serves as the gold target for decomposition MAPE.

Scenario-spec example

```yaml
# helios_eval/scenarios/S017_paid_search_mobile_rate_drop.yaml
scenario_id: S017
title: "Paid Search x mobile checkout-to-purchase rate collapse"
anomaly_type: rate            # mix | rate | mixed | none
seed: 1759                    # deterministic injector seed
baseline_window:  { start: "2020-12-01", end: "2020-12-20" }
inject_at:        "2020-12-21"
eval_window:      { start: "2020-12-21", end: "2021-01-10" }
target_metric:    checkout_to_purchase_rate
funnel_step:      { numerator: purchasing_sessions, denominator: begin_checkout_sessions }
perturbation:
  dimension: [channel_group, device_category]
  segment:   { channel_group: "Paid Search", device_category: "mobile" }
  rate_multiplier: 0.55       # 45% relative rate drop in this cell only
expected_ground_truth:
  root_cause_segment: { channel_group: "Paid Search", device_category: "mobile" }
  dominant_effect: rate       # graded against Decompose output
  is_seasonality_decoy: false
  dollar_at_risk_usd: null    # filled by injector post-materialization
controls:
  hold_constant: [sessions]   # volume fixed -> isolates rate effect
```

20.3 The eval dataset (>= 30 scenarios)

The benchmark ships **34 scenarios** spanning the required coverage axes. Each must be reproducible from its seed.

Bucket	Count	What it tests
Single-segment rate-change	6	One dimension cell's step rate moves; Decompose must attribute to **rate** effect and pin the segment.
Single-segment mix-shift	6	One segment's `sessions` share moves; must attribute to **mix** effect (Simpson's-paradox guard).
Multi-segment rate-change	4	Several cells move together; top-3 must contain all true cells.
Multi-segment mixed (mix+rate+interaction)	4	Both `w_i` and `r_i` move; tests interaction-term handling.
Seasonality decoys	4	A *real* seasonal swing (e.g. post-holiday week-over-week dip) is present but is **not** the injected anomaly; system must NOT flag it (Critic must refute).
No-anomaly controls	6	Zero perturbation; tests false-positive rate of Monitor (`detect_anomaly`).
Data-quality / confound	4	Injected NULL spikes, transaction_id duplication, late-arriving shard; Critic must catch as data quality, not behavior.

20.4 Metrics

Each metric below is computed by the harness in deterministic Python (never the LLM) and aggregated across scenarios.

Metric	Definition	Target
Root-cause segment accuracy (top-1)	fraction of scenarios where Diagnose's #1 root-cause segment == labeled segment	**>= 85%**
Root-cause segment accuracy (top-3)	labeled segment appears in Diagnose's top-3 ranked candidates	>= 95%
Decomposition error (MAPE)	mean abs % error between Decompose's estimated mix/rate/interaction contributions and analytic ground truth	<= 10%
Anomaly precision / recall / F1	over all `(scenario, day, metric)` cells flagged by Monitor vs labeled injections; controls contribute to precision	F1 >= 0.85

Metric	Definition	Target
Dollar-at-risk estimation error	abs % error of estimated revenue-at-risk vs label `dollar_at_risk_usd`	<= 15%
Hallucination rate	any column/metric in emitted SQL not present in the semantic-mcp registry or GA4 schema (AST-checked)	**0%**
Faithfulness	does Narrator's prose claim match the SQL evidence + stats-mcp outputs (entailment check)	>= 0.95

Top-1 accuracy is the headline. **Hallucination rate** is a hard gate (any non-zero value fails CI regardless of accuracy) because grounding-over-generation is a first principle.

20.5 Naive baseline ("largest absolute segment delta")

The baseline a competent-but-unsophisticated analyst would use: for the anomalous metric, compute each segment's `delta = value_at(t1) − value_at(t0)`, rank by `|delta|`, and declare the single largest-magnitude segment the root cause. It performs **no mix-vs-rate decomposition**, so it is systematically fooled by mix-shift (a high-volume segment whose *rate* barely moved still shows the largest absolute delta). On our 34-scenario benchmark this baseline scores **~45% top-1** (it gets pure single-segment rate cases right and most mix cases wrong). Helios must clear **85%**, a near-doubling, which is the project's central empirical claim.

20.6 Harness architecture

```
helios/eval/
  injector.py        # seeded perturbation → fct_daily_funnel_perturbed + labels
  runner.py          # for each scenario: point pipeline at perturbed copy, run all 7 agents
  scorers/
    rootcause.py     # top-1 / top-3 segment accuracy
    decomposition.py # MAPE on mix/rate/interaction
    detection.py     # precision/recall/F1 for Monitor
    dollars.py       # dollar-at-risk error
    hallucination.py # SQL AST vs semantic-mcp registry + GA4 schema
    faithfulness.py  # narrative⟷evidence entailment (Critic-as-judge + rule checks)
  report.py          # aggregates → results table + per-scenario JSON + markdown
  scenarios/*.yaml   # the 34 specs
```

```python
# runner.py (core loop, abbreviated)
def run_benchmark(scenarios, pipeline):
    results = []
    for spec in scenarios:
        inject(spec)                                  # materialize perturbed copy + label
        ctx = PipelineContext(dataset="helios_eval",
```

```
                                table="fct_daily_funnel_perturbed",
                                window=spec.eval_window)
            diag = pipeline.run(ctx)                        # Orchestrator..Narrator+Critic
            label = load_label(spec.scenario_id)
            results.append(score_all(diag, label, scorers))
        return aggregate(results)                            # → results table
```

The harness pins random seeds, freezes the dbt model SHA, and records the semantic-mcp registry hash so a run is fully reproducible. Each scenario emits a per-scenario JSON artifact (predicted vs label, every sub-score) for debugging.

20.7 Scoring details

- **Root-cause matching** is on the normalized segment key (sorted dimension=value pairs). A predicted segment matches the label iff dimension set and values are equal; partial matches (right dimension, wrong value) count as miss for top-1.
- **Decomposition MAPE** is computed only on scenarios where a true effect exists (controls excluded), comparing the three contribution buckets element-wise, then averaging.
- **Faithfulness** runs two checks: (1) a rule check that every numeric claim in the brief has a backing `run_query` result hash and a `significance_test` p-value attached; (2) a Critic-as-judge entailment pass that flags any sentence not entailed by the evidence bundle. Both must pass.

20.8 Regression gating in CI

The benchmark runs in **GitHub Actions** as a required check on every PR that touches `models/`, `semantic/`, `agents/`, or `eval/`. Gating thresholds (stored in `eval/gates.yaml`):

```
gates:
  rootcause_top1_min: 0.85
  decomposition_mape_max: 0.10
  hallucination_rate_max: 0.00     # hard zero
  detection_f1_min: 0.85
  dollar_error_max: 0.15
  faithfulness_min: 0.95
  regression_tolerance: 0.02       # top1 may not drop >2pts vs main baseline
```

CI fails the PR if any gate is breached OR if top-1 regresses more than `regression_tolerance` against the committed `main` baseline (`eval/baselines/main.json`). To bound cost (Foundation byte-budget target), the harness runs every scenario through `warehouse-mcp.dry_run` first and aborts if total scanned bytes exceed the per-run budget; a 12-scenario **smoke subset** runs on every push, the full 34 on PRs to `main`.

20.9 Results-table template

```
Helios Eval Report — run 2026-06-03  model_sha=ab12cd  registry=9f3e
═══════════════════════════════════════════════════════════════════
Metric                          Helios    Baseline   Target   Pass
───────────────────────────────────────────────────────────────────
Root-cause top-1                0.882     0.441      ≥0.85    PASS
Root-cause top-3                0.971     0.618      ≥0.95    PASS
Decomposition MAPE              0.073     n/a        ≤0.10    PASS
Anomaly detection F1            0.901     0.560      ≥0.85    PASS
Dollar-at-risk error            0.118     n/a        ≤0.15    PASS
Hallucination rate              0.000     0.000      ==0.00   PASS
Faithfulness                    0.962     n/a        ≥0.95    PASS
───────────────────────────────────────────────────────────────────
Scenarios: 34   Cost: 2.1 GB scanned (budget 5 GiB (~5.37 GB))   Time: 4m12s/run
```

This table is rendered to the PR comment and archived under `eval/history/`, giving a longitudinal record of whether the 85%-vs-45% claim continues to hold as the system evolves.

---

21. Experimentation Framework

Helios does not stop at diagnosis. The **Prescribe** agent turns each surviving finding into a **statistically-defensible experiment design**: a hypothesis card, a powered sample size, a runtime estimate, and a prioritization score. The honest constraint stated up front: **this GA4 dataset is OBSERVATIONAL and historical — there is no live traffic to A/B test against.** Helios therefore (a) *designs and sizes* experiments a team could run, and (b) for already-occurred changes, runs **quasi-experimental readbacks** (pre/post and difference-in-differences) using `stats-mcp`. Every prescribed experiment ties back to a `dollar-at-risk` from the diagnosis, so the backlog is ranked by money.

21.1 Hypothesis-card schema

Each prescription is a governed object produced by `experiment-mcp.design_experiment(hypothesis, metric)` and persisted via `report-mcp.save_diagnosis`.

```yaml
hypothesis_card:
  card_id: H-2021-0042
  source_finding_id: F-2021-0042          # link to the diagnosis it came from
  hypothesis: >
    Because mobile begin_checkout_sessions on Paid Search convert at a 45%
    lower checkout_to_purchase_rate post 2020-12-21, simplifying the mobile
    payment step will recover purchasing_sessions.
  target_metric: checkout_to_purchase_rate  # canonical metric name
```

```yaml
  segment: { channel_group: "Paid Search", device_category: "mobile" }
  expected_mechanism: "reduce form friction at add_payment_info → purchase"
  variant_description: "one-tap wallet + autofill on mobile checkout"
  primary_metric: checkout_to_purchase_rate
  guardrail_metrics: [aov, cart_abandonment_rate, revenue_per_session, net_revenue]
  baseline_rate: 0.061                    # observed in control segment
  mde_relative: 0.10                      # detect a 10% relative lift
  alpha: 0.05
  power: 0.80
  test: two_proportion_z
  sample_size_per_arm: null               # filled by power_analysis
  runtime_days: null                      # filled by runtime_estimate
  ice_score: null
  lifecycle_state: proposed
```

21.2 Statistics

All math is delegated to `stats-mcp` and `experiment-mcp`; the LLM only supplies parameters. For a binomial primary metric (a conversion rate at a funnel step), the required sample size per arm for a **two-proportion z-test** is:

```
p1 = baseline_rate
p2 = baseline_rate * (1 + mde_relative)          # treatment under H1
pbar = (p1 + p2) / 2
n_per_arm = ( z_(1-alpha/2) * sqrt(2*pbar*(1-pbar))
            + z_(1-beta)   * sqrt(p1*(1-p1) + p2*(1-p2)) )^2
            / (p2 - p1)^2
```

with `z_(1-alpha/2)=1.96` at alpha=0.05 (two-sided) and `z_(1-beta)=0.84` at power=0.80. `experiment-mcp.power_analysis(baseline, mde, alpha, power)` returns `n_per_arm`; `runtime_estimate(n, traffic)` divides total required n (`2 * n_per_arm`) by the **observed eligible traffic rate** for the segment (sessions/day reaching the funnel step) to yield `runtime_days`.

```python
# worked example for card H-2021-0042
p1 = 0.061; mde = 0.10
n = power_analysis(baseline=p1, mde=mde, alpha=0.05, power=0.80)["n_per_arm"]
# ~ 14,800 begin_checkout_sessions per arm
traffic = run_query("select avg(begin_checkout_sessions) "
                    "from fct_daily_funnel "
                    "where channel_group='Paid Search' and device_category='mobile'")
# observed ~ 95 begin_checkout_sessions/day in this segment
runtime = runtime_estimate(n_total=2*n, traffic_per_day=95)  # ~ 311 days → UNDERPOWERED
```

This example deliberately surfaces the dataset's core limitation: thin per-segment traffic makes many fine-grained tests impractically long. The Prescribe agent must report `runtime_days`

honestly and, when it exceeds a threshold, recommend **rolling the test up to a coarser segment** (e.g. all `mobile` rather than `Paid Search x mobile`) or relaxing `mde_relative`, re-sizing, and noting the trade-off.

Sequential / peeking caveats

Naively checking a running test repeatedly inflates the false-positive rate far above alpha. Helios designs are **fixed-horizon by default** (decide n up front, read once at `runtime_days`). When continuous monitoring is desired, `experiment-mcp.design_experiment` applies an **alpha-spending / group-sequential** correction (O'Brien-Fleming-style boundaries) or specifies an **always-valid sequential test** (mSPRT), and the card records the chosen procedure so the readout uses the matching stopping rule. The Critic explicitly checks every readout for peeking violations.

Multiple-comparison handling

A single run can generate many hypothesis cards. When several primaries are tested in one program, Helios applies **Benjamini-Hochberg FDR control** across the family of p-values (preferred for backlog screening) or Bonferroni for a small confirmatory set, and records the adjusted threshold on each card. Guardrail metrics are evaluated as one-sided non-inferiority checks at their own alpha.

21.3 Mapping onto the GA4 dataset

Testable primary metric	Funnel step	Typical daily volume (store-wide)	Testability
view_to_cart_rate	view_item -> add_to_cart	high (thousands of view_item_sessions/day)	strong — fast tests
cart_to_checkout_rate	add_to_cart -> begin_checkout	medium	moderate
checkout_to_purchase_rate	begin_checkout -> purchase	low (hundreds/day store-wide)	weak at segment grain
session_conversion_rate	sessions -> purchasing_sessions	medium denominator, low numerator	moderate store-wide, weak by segment
aov / revenue_per_session	continuous, per transaction	low transaction count	weak (high variance)

Rule of thumb encoded in Prescribe: tests on **upper-funnel rate metrics** (view_to_cart_rate) are well-powered within weeks; **lower-funnel and revenue metrics** are usually only powerable at coarse grain or store-wide. Continuous metrics (`aov` , `revenue_per_session`) use a Welch t-test (via `significance_test`) rather than the z-test and need variance estimates from the data.

21.4 Prioritization model (ICE / PIE)

Helios scores every card with **ICE** and ranks the backlog descending:

```
Impact     = dollar_at_risk (from diagnosis)  x  expected_relative_lift (mde or modeled)
Confidence = evidence strength in [0,1]
             = f(significance p-value, decomposition cleanliness, sample adequacy,
                 Critic-survival, prior-similar-finding outcomes from memory)
Effort     = engineering estimate in [1..10] (variant complexity + instrumentation)

ICE_score  = (Impact_normalized * Confidence) / Effort
```

Impact is grounded in **real dollars** because every finding carries a `dollar-at-risk` , so the backlog is literally sorted by recoverable revenue per unit effort. `Confidence` downweights cards whose `runtime_days` is impractical or whose decomposition had high interaction (ambiguous attribution). A PIE variant (Potential, Importance, Ease) is available as an alternate weighting; both are deterministic and recorded on the card for auditability.

21.5 Experiment lifecycle and memory tracking

```
proposed ──> designed ──> running ──> readout ──> (archived: won | lost | inconclusive)
   |            |            |           |
 Prescribe   power+runtime  team runs   stats-mcp readback
 emits card  sized & ICE'd  (external)  + Critic verifies
```

- **proposed**: Prescribe creates the card from a surviving finding.
- **designed**: `power_analysis` + `runtime_estimate` fill `sample_size_per_arm` and `runtime_days` ; ICE computed; Critic checks the design (powered? guardrails sensible? peeking rule set?).
- **running**: external to Helios (no live traffic here) — state tracked only.
- **readout**: Helios computes results. For real online tests, `significance_test(a,b)` . For this **observational dataset**, Helios runs **quasi-experimental readbacks**: a **pre/post** comparison of the segment around `inject_at` , and a **difference-in-differences** that nets out a comparable control segment to remove seasonality (`DiD = (treated_post - treated_pre) - (control_post - control_pre)`), with the parallel-trends assumption checked on the pre-period and reported. The Critic attempts to refute (confounding, non-parallel pre-trends, mix-shift in the control).

Every state transition and result is persisted by `report-mcp.save_diagnosis` into the **Memory store**, and `report-mcp.recall_prior(metric, segment)` lets future runs see whether a similar experiment already won or lost — feeding the `Confidence` term and preventing the backlog from re-proposing settled questions. This closes the loop from autonomous diagnosis to a living, money-ranked, statistically-defensible experiment backlog.